PIRTM (Prime-Indexed Recursive Tensor Mathematics): Beginner's Manual

What PIRTM is

PIRTM is a compiler-and-runtime project that treats *session identity* and *stability proofs* as first-class compile artifacts rather than runtime best-effort checks. The system is being specified as an MLIR dialect (`pirtm`) with explicit type-level identity (`mod=` parameters) and verifier passes that make certain unsafe programs syntactically invalid, rather than merely "discouraged."

A practical mental model is "a programming language toolchain where every build produces cryptographically anchored proof metadata and where network-level stability is checked at link time, not at runtime."

What PIRTM does (high level)

PIRTM's toolchain is organized around a two-phase compilation model: per-session transpile produces `.pirtm.bc` modules with local contractivity proofs, and a later link step produces a sealed runtime binary after verifying a network small-gain condition over a coupling matrix Ψ. This split mirrors the idea that some correctness obligations are local (a single module) while others only make sense globally (a network of interacting modules).

PIRTM also explicitly separates **offline-inspectable proof metadata** from **runtime audit telemetry**, and introduces dedicated tools (`pirtm inspect` and `pirtm audit`) to prevent operators from confusing "designed safely" with "ran safely."

Beginners' glossary

- **Prime channel / prime_index**: A canonical numeric identity for an atomic session/module, treated as the basis element for network objects like Ψ.
- **identity_commitment**: A human-authored or build-authored commitment/alias carried with the module, used for cross-checking and chain-of-custody, but not used as the canonical index for linear-algebraic objects.
- **mod= parameter**: The canonical identity carried by tensor *types* in the dialect (atomic: prime mod; composite: squarefree mod).
- **Contractivity proof**: A local condition checked at transpile time for a single module, parameterized by scalar ε and an operator norm bound op_norm_T.
- **Network ISS proof**: A global condition checked at link time, using a gain matrix Ψ and requiring spectral radius r(Ψ) < 1 ("spectral small-gain").

The compilation model (transpile → link → runtime)

Transpile phase: local proofs

During **transpile**, each session compiles to its own .pirtm.bc containing exactly one pirtm.module with scalar parameters: prime_index, epsilon, op_norm_T, and identity_commitment. This phase runs verifier passes that ensure prime validity and squarefree validity for types, enforce coprime merge rules, and perform contractivity-check on step operations using ε from the parent module.

The key governance invariant is strict: one module per prime, no epsilon_map, no multi-prime modules.

Link phase: global stability

During **link**, the toolchain builds a pirtm.session_graph with a fully resolved coupling matrix Ψ and checks r(Ψ) < 1 using a spectral-small-gain verifier pass. This check is explicitly link-time only; before

linking, the module carries a sentinel `#pirtm.unresolved_coupling` to prevent premature verification.

Coupling is authored in a `coupling.json` file that uses human-friendly session names, but the linker resolves those names into canonical prime identities before constructing Ψ.

Runtime phase: execution, not proof

At runtime, PIRTM is designed to execute a sealed binary; it does not "re-prove" the properties that were already certified at compile/link time. Runtime produces *audit traces* (telemetry, certificate production/consumption records), but those are treated as execution artifacts rather than proof artifacts.

Mathematical overview (conceptual, beginner-friendly)

1) Prime-indexed identity as a basis

PIRTM's network-level stability object is the gain matrix Ψ, and its meaning depends on having a well-defined index set (basis) for rows and columns. PIRTM's design chooses prime identities (`prime_index`) as the canonical basis labels so Ψ is a linear-algebraic object over a stable, unique index set.

In beginner terms: Ψ is "a table of how much session B can affect session A," but to compute properties like spectral radius you must be unambiguous about what each row/column *is*. Using a stable numeric identity makes this explicit.

2) Local contractivity vs global small-gain

PIRTM treats stability as two different obligations:

- **Local (per module)**: the module has a contractivity bound parameterized by ε and norms captured in `op_norm_T` (plus internal norms like $\|\Xi\|$ and $\|\Lambda\|$ implied by the design).
- **Global (network)**: the composed system is stable if the coupling gains satisfy $r(\Psi) < 1$ (spectral radius less than one), which is checked at link time.

This separation matters because a system can be locally stable in each component but still unstable when coupled strongly; conversely, weak coupling can preserve stability.

3) Squarefree composite channels and CRT intuition

PIRTM introduces composite tensor types where the `mod=` parameter must be **squarefree** (no repeated prime factor), which corresponds to the idea that composite identities behave like products of distinct prime channels. The ADR frames this as structurally exposing the CRT-style decomposition rather than hiding it in runtime representations.

In beginner terms: combining channels is allowed only when the identity factors are "clean" (no repeated factor), so projections and merges are deterministic and safe by construction.

4) Why link-time Ψ is non-negotiable

The link step is where the system first has a complete view of *all* participating modules and their interconnections, so that is the earliest moment when `spectral-small-gain` is meaningful. Putting Ψ into per-module metadata at transpile time would either be incomplete (missing other sessions) or would require global knowledge during transpile, defeating the modular design.

Identity: two-layer and three-layer separations

Two-layer identity (canonical vs human-authored)

Coupling authoring is treated as a usability problem: humans want names, math wants canonical indices. PIRTM resolves this with a two-layer scheme:

- Human layer: `coupling.json` names are ergonomic and editable.
- Canonical layer: the linker resolves each name to a `prime_index` and validates `identity_commitment` matches the `.pirtm.bc` module metadata.

After linking, the pirtm.session_graph is prime-indexed; names do not survive into IR.

Three-layer audit architecture (static proof vs governance vs runtime audit)

PIRTM's documentation explicitly distinguishes three layers:

1. Static proof metadata embedded in .pirtm.bc (offline readable).
2. Link-time governance metadata embedded in the sealed binary (offline readable).
3. Runtime audit chain emitted during execution (trace-based; not embedded).

This prevents an unsound inference: a clean runtime trace does not imply the universal claims of a static contractivity proof, and static proof does not describe a particular execution trajectory.

What a beginner workflow looks like

1. **Transpile**: produce one .pirtm.bc per session/module, which includes the module's prime identity, ε, op_norm_T, and proof metadata.
2. **Inspect (offline)**: run pirtm inspect on .pirtm.bc to retrieve identity and static proof fields without executing anything.
3. **Author coupling**: write coupling.json using session names and gains; the file also includes prime_index and commitment bindings per name.
4. **Link**: run pirtm link --coupling coupling.json to resolve names → primes, cross-check commitments, construct Ψ, and verify r(Ψ) < 1 before sealing a runtime binary.
5. **Run and audit**: execute the sealed binary to produce traces; run pirtm audit <trace.log> --binary <sealed> to cross-reference the runtime trace against the static proof hash (chain-of-custody).

Realistic implications (what this enables, what it does not)

What is realistically strong

- **Earlier failure modes**: Many stability and identity errors become compile/link-time errors rather than runtime surprises, because proofs are enforced as verifier passes and link gates.
- **Reproducible provenance**: Embedding proof and governance metadata enables offline inspection and chain-of-custody checks (especially when the proof hash is content-addressable).
- **Modular scaling**: The two-phase model (per module transpile, later link) supports many sessions compiled independently but verified together at seal time, similar to link-time optimization in other toolchains.

What remains hard / not guaranteed

- **Choosing Ψ values**: The off-diagonal gain values in `coupling.json` are domain knowledge and cannot be inferred automatically in general; they are the "human-authored hard part."
- **Global stability is brittle to new coupling**: Adding a single new session can invalidate $r(\Psi) < 1$; the ADR explicitly notes that $r(\Psi')$ is not monotone with respect to adding a session.
- **Runtime audit ≠ static proof**: A clean audit chain is evidence of one execution's behavior, not a universal correctness statement about the program's design; PIRTM's three-layer architecture is meant to prevent this confusion, but operators must still follow process.

Future outlook (near-term roadmap and why it's plausible)

PIRTM's near-term plan is framed as a sequence of verifiable gates rather than a large "big bang" implementation. The Day 0–3 gate is a minimal MLIR dialect type layer with prime/squarefree verifiers and a four-line `--verify-diagnostics` test, which is a realistic way to de-risk the type/identity foundation early.

Mid-term (Day 14–30) focuses on hardening the toolchain boundary: `pirtm inspect` must confirm static proof metadata from `.pirtm.bc`, and `pirtm link` must enforce coupling resolution and spectral-small-gain prior to sealing binaries.

Longer-term (Day 90) moves into performance and runtime engineering (Rust kernel, PyO3 bindings) only after the proof-carrying compilation pipeline is stable, which is a realistic dependency order for safety-first systems.

How this relates to MLIR (context for beginners)

MLIR is designed around extensible dialects that define operations, types, attributes, and verifiers from declarative TableGen records, reducing boilerplate and making the dialect definition a "single source of truth" for many generated utilities. This aligns with PIRTM's approach of encoding safety invariants into the IR itself so they can be verified mechanically by passes and toolchain stages, rather than being enforced ad hoc at runtime.[1][2]

Appendix: Key objects and where they live

Object	Where it appears	Purpose
`pirtm.module`	`.pirtm.bc`	Carries `prime_index`, `epsilon`, `op_norm_T`, `identity_commitment`; enforces flat governance.
`pirtm.session_graph`	sealed binary (post-link)	Carries prime-indexed `session_primes`, resolved Ψ, debug commitments; runs spectral small-gain at link time.
`coupling.json`	input to link	Human-authored coupling gains and name→(prime,commitment) table; resolved by linker passes.
`pirtm inspect`	CLI	Offline reader for static proof + governance metadata; must clearly state audit chain is not embedded.
`pirtm audit`	CLI	Post-run verification tool that reads trace logs and cross-references the binary's proof hash (chain-of-custody).

References

1. llvm-project/mlir/docs/DefiningDialects/Operations.md at main · llvm/llvm-project - The LLVM Project is a collection of modular and reusable compiler and toolchain technologies. - llvm...
2. Defining Dialects - MLIR - Multi-Level IR Compiler Framework